

eTCP – Part 2

Reliable Data Transfer + Flow Control + Congestion Control

1. Overview

In Part 1, you implemented TCP connection management: connection setup and teardown. Once two endpoints reach the ESTABLISHED state, however, the work of TCP has only just begun.

A correct transport protocol must now ensure that data is:

- delivered reliably,
- delivered in order,
- acknowledged correctly,
- retransmitted when lost,
- and transmitted only as fast as both the receiver and the network can handle.

In Part 2, you will implement the core data-path logic of TCP:

- Reliable data transfer (RDT)
- Flow control
- Congestion control

As in Part 1, the protocol logic is split across the receive path and the send path. On the receive side, the eTCP thread processes incoming data packets and ACKs, updates per-flow state, merges payload into the receive buffer, and reacts to loss or duplicate acknowledgments. On the send side, the eTCP thread constructs outgoing TCP segments, enforces send-window limits, and transmits payload data from the send buffer.

2. Reliable Data Transfer Overview

TCP provides a reliable byte-stream service on top of an unreliable IP network. This means that even if packets are lost, duplicated, or arrive out of order, the application should still observe a correctly ordered stream of bytes.

To achieve this, TCP relies on:

- Sequence numbers to identify every byte in the stream
- Acknowledgments (ACKs) to tell the sender what data has been received
- Retransmissions to recover lost data
- Buffers on both the sender and receiver sides
- Timers to detect missing acknowledgments

The sender keeps data in a send buffer until it is acknowledged. Important variables include:

- `snd_una`: the oldest unacknowledged sequence number

- `snd_nxt`: the next sequence number to send

The difference (`snd_nxt - snd_una`) represents the amount of in-flight data, i.e., data that has been sent but not yet acknowledged.

The receiver stores incoming payload in a receive buffer. Important variables include:

- `rcv_nxt`: the next sequence number expected in order
- `rcv_wnd`: currently available receive-buffer space

If data arrives in order, the receiver can advance `rcv_nxt` immediately. If data arrives out of order, the receiver must buffer it and wait until the missing earlier bytes arrive. If the sender does not receive the necessary ACK in time, it must retransmit the missing segment. This can happen in two ways:

- by timeout
- by fast retransmission, triggered by duplicate ACKs

3. Flow Control Overview

- What is Flow Control?

Flow control prevents a fast sender from overwhelming a slow receiver. Even if the network is fast and loss-free, the receiver may not be able to process incoming data quickly enough. For example:

- The receiving application may not be reading fast enough
- The receiver's TCP buffer may be nearly full

To prevent buffer overflow, TCP receivers advertise the amount of free space in their receive buffer. This value is called the advertised window (often denoted as `rwnd`). The sender must never send more data than the receiver can currently accept.

- Receiver-Advertised Window

Each ACK packet in TCP contains a window field saying that "I can currently accept this many more bytes." If the receiver buffer becomes full, the window may shrink to zero. In that case, the sender must stop sending new data until the receiver advertises a larger window again. This mechanism is called receiver-based flow control.

- Sender View of Flow Control

From the sender's point of view:

- `snd_una` = oldest unacknowledged sequence number
- `snd_nxt` = next sequence number to send

- in-flight data = $\text{snd_nxt} - \text{snd_una}$
- peer-advertised window = receiver's available buffer space

Then the sender may transmit more data only if ($\text{in-flight} < \text{advertised_window}$).

- Why Flow Control Matters

Without flow control:

- The sender may overflow the receiver's buffer
- Packets may be dropped unnecessarily
- The receiver may lose data even if the network itself is not congested

So flow control is not about the network becoming busy. It is specifically about the receiver's ability to absorb data.

4. Congestion Control Overview

- What is Congestion Control?

Congestion control prevents the sender from injecting too much traffic into the network. Even if the receiver has plenty of buffer space, the network may still become congested because:

- Routers have limited queue space
- Links have limited bandwidth
- Excessive traffic may cause packet loss and delay

- Congestion Window (cwnd)

TCP uses another sender-side variable called the congestion window (cwnd). Unlike the advertised window, cwnd is not directly told by the receiver. Instead, it is maintained by the sender based on network behavior such as:

- successful ACKs
- packet loss
- retransmission timeout
- duplicate ACKs

The sender treats cwnd as the current safe amount of in-flight data allowed by the network.

- Slow Start

At the beginning of a connection, the sender does not know the available network capacity. Therefore it starts conservatively with a small congestion window and increases it rapidly as ACKs arrive.

If $\text{cwnd} < \text{ssthresh}$, the sender is in slow start. In this phase, cwnd increases aggressively as

new ACKs arrive. As a result, the sending rate grows quickly and probes for available bandwidth.

- Congestion Avoidance

Once $cwnd$ reaches $ssthresh$, the sender switches to congestion avoidance.

If $cwnd \geq ssthresh$, the sender is in congestion avoidance. In this phase, $cwnd$ still grows on new ACKs, but much more slowly than in slow start. This allows the sender to probe for additional bandwidth more carefully.

- Three Duplicate ACKs: Fast Retransmit / Fast Recovery

If the sender receives three duplicate ACKs, TCP Reno assumes that a packet was lost but that the network is still delivering later packets.

In this case:

- the sender retransmits the missing segment immediately without waiting for a timeout,
- reduces its sending rate,
- and enters fast recovery.

A typical Reno-style reaction is:

- reduce $ssthresh$,
- temporarily inflate $cwnd$ during recovery,
- and then return to congestion avoidance after recovery completes.

This is less severe than a retransmission timeout, because duplicate ACKs indicate that packets are still flowing through the network.

- Retransmission Timeout (RTO)

A retransmission timeout is treated as a stronger congestion signal than duplicate ACKs.

If an RTO occurs:

- the sender assumes congestion is severe,
- reduces $ssthresh$,
- resets $cwnd$ to a very small value,
- and restarts from slow start.

Thus, TCP Reno reacts more aggressively to timeouts than to three duplicate ACKs.

5. Reliable Data Transfer in eTCP

In eTCP, reliable data transfer is implemented across both `tcp_in.c` and `tcp_out.c`. On the receive side, incoming TCP segments are processed in the TCP input path. On the send side, the buffered payload is segmented and transmitted by the TCP output path.

- ACK Processing in eTCP: *HandleACK*(in `tcp_in.c`)

The central ACK-processing function in part 2 is *HandleACK* in `tcp_in.c`. This function handles incoming ACK segments after the connection has been established.

Conceptually, *HandleACK* performs several jobs at once:

- Validate the ACK
- Update flow-control state
- Detect duplicate ACKs
- Handle fast retransmission
- Remove newly acknowledged data from the send buffer
- Update sender-side state variables
- Optionally update congestion-control variables

In *HandleACK*, before using the incoming ACK, the function checks whether the ACK number is outside the valid send-buffer range. If an ACK acknowledges data beyond what is currently buffered, it is ignored. This prevents invalid or stale packets from corrupting sender-side state.

- Send Buffer Advancement

If the ACK newly acknowledges data, *HandleACK* calculates the number of acknowledged bytes ($rmlen = ack_seq - sndbuf->head_seq$). Then it removes those bytes from the send buffer, advances `snd_una`, and updates the amount of available sender-side buffer space. This is one of the most important parts of reliable data transfer. A sender must not discard data until it has been acknowledged.

- Receiver-Side Data Processing

Incoming payload is handled on the receive side by the *HandlePayload* function in `tcp_in.c`. The receiver must:

- validate that the payload falls within the current receive window,
- insert the payload into the receive buffer,
- support out-of-order arrival,
- advance `rcv_nxt` only when data becomes contiguous,
- update `rcv_wnd`,
- queue an ACK.

This is the receiver half of reliable data transfer.

- Packet Construction on the Sender Side

Outgoing packets are built by *MakeTCPPacket* in *tcp_out.c*. In part 1, you have implemented the construction of control packets carrying no payload. Here, you must implement the construction of a packet that does carry payload. Important additions are:

- payload data is copied into the packet
- *snd_nxt* is updated when data is sent
- The retransmission-related timing state is updated
- The current receive window is advertised in the outgoing header

- Sending Buffered Data

Buffered application data is transmitted through *ProcessSendBuffer* in *tcp_out.c*. This function repeatedly takes data from the send buffer and turns it into TCP segments, as long as transmission is allowed.

At a high level, it performs the following steps:

- Determine the first unsent byte using *snd_nxt*
- Translate that sequence number into an offset inside the send buffer
- Compute how much buffered data remains
- Compute how much data may be sent right now
- Limit the segment by MSS and window constraints
- Call *MakeTCPPacket* to actually transmit the segment

This function is where reliable transmission, flow control, and congestion control come together.

6. Flow Control in eTCP

In eTCP, flow control is implemented through both:

- receiver-side window calculation
- sender-side enforcement of the peer's window

- Receiver Side: Updating *rcv_wnd* (in *HandlePayload* in *tcp_in.c*)

When payload is received and inserted into the receive buffer, eTCP updates *rcv_nxt* and *rcv_wnd*. (*rcv_nxt* = next sequence number expected in order, *rcv_wnd* = remaining free space in the receive buffer). This means the receiver always tracks how much more data it can accept. Later, when eTCP sends an ACK, it places *rcv_wnd* into the TCP header window field, thereby advertising the receiver's available buffer space to the peer.

So in eTCP:

- Receive buffer occupancy determines *rcv_wnd*

- rcv_wnd is advertised to the sender through ACK packets
- Sender Side: Tracking peer_wnd(in *HandleACK* in tcp_in.c)

When ACKs arrive, eTCP reads the advertised window from the incoming TCP header and stores it in `sndvar->peer_wnd`. This variable represents the receiver's currently advertised window. However, ACKs may arrive out of order or be duplicated, so eTCP does not blindly overwrite `peer_wnd` every time. Instead, it uses bookkeeping variables such as *snd_wl1* and *snd_wl2* (explained in the skeleton code) to determine whether the incoming window update is newer than the previously accepted one. This prevents stale ACKs from corrupting flow-control state.

- Sender Side: Deciding How Much Data to Send(in *ProcessSendingBuffer* in tcp_out.c)

When eTCP flushes buffered application data, it computes how much data is currently allowed to leave the sender. So, even if the send buffer contains lots of data, eTCP sends only the amount allowed by both the network and the receiver. This logic appears in the sending path, especially in the function that flushes the sending buffer into TCP segments.

- Reopening the Send Path

If the receiver window was previously too small, the sender may become blocked. Later, when a new ACK advertises a larger window, eTCP detects that sending is possible again and raises a write event so the application can continue writing. This is important for non-blocking socket semantics and epoll-based designs.

7. Congestion Control in eTCP

In this project, you will implement a simplified Reno-style congestion control mechanism in eTCP. The goal is not to reproduce every detail of a production TCP stack, but to implement the core behavior correctly and consistently.

- Initial Settings

Use the following initial values after connection establishment:

- $cwnd = TCP_INIT_CWND * MSS$
- $ssthresh = 10 * MSS$

Your implementation should then update these values according to ACKs, duplicate ACKs, and retransmission timeouts.

- Behavior You Should Implement

1. Slow Start

The sender begins in slow start.

- If $cwnd < ssthresh$, the sender is considered to be in slow start.
- On each new ACK that advances snd_una , increase $cwnd$ aggressively.
- In this simplified implementation, slow start should increase $cwnd$ by approximately 1 MSS per advancing ACK.

2. Congestion Avoidance

When the congestion window grows large enough, the sender should enter congestion avoidance.

- If $cwnd \geq ssthresh$, use congestion avoidance.
- On each new ACK that advances the acknowledgment number, increase $cwnd$ more conservatively than in slow start.

3. Three Duplicate ACKs

If the sender observes three duplicate ACKs, it should assume that a segment has been lost and perform fast retransmit / fast recovery.

At that point, the sender should:

- retransmit the missing segment immediately without waiting for timeout,
- set $ssthresh$ and $cwnd$ appropriately
- enter a fast recovery state.

While in fast recovery:

- each additional duplicate ACK should inflate the congestion window by 1 MSS,
- the sender may transmit more data if allowed by the effective sending window.

When a new ACK arrives that acknowledges forward progress, the sender should:

- leave fast recovery,
- deflate the congestion window,
- continue in congestion avoidance.

4. Retransmission Timeout

If a retransmission timeout (RTO) occurs, the sender should react more conservatively than in the three-duplicate-ACK case. It should

- update $ssthresh$
- reset $cwnd$
- leave fast recovery if it was active
- retransmit starting from the first unacknowledged byte.

After timeout recovery, the sender should resume transmission from slow start.

8. What should I do?

Now you are ready to fill in the TCP data-transfer logic. Source code files you need to update are:

- tcp_in.c
- tcp_out.c
- timer.c

- tcp_in.c

1. *HandleACK*

This is the core receive-side function for sender-state updates. Your implementation should handle the following:

(a) Update Peer Advertised Window

When an ACK arrives, update `peer_wnd` only if the incoming ACK contains a newer window value than the previous update. This is part of flow control.

(b) Detect Duplicate ACKs

A duplicate ACK should not be identified only by checking whether the ACK number is the same as before. In this project, treat an ACK as a duplicate ACK only when all of the following are true:

- the ACK number does not advance the sender state
- the segment carries no payload
- the segment does not provide a meaningful new window update

In other words, a duplicate ACK is a pure ACK that repeats the same acknowledgment point without newly acknowledging data or changing the effective advertised window. This distinction is important because not every ACK with the same ACK number should be counted toward fast retransmission

(c) Fast Retransmission

If three duplicate ACKs are observed, schedule retransmission immediately rather than waiting for the timeout.

(d) Remove Acknowledged Bytes

Once an ACK newly acknowledges data, remove those bytes from the send buffer, advance `snd_una`, and refresh the sender-side writable state.

(e) Wake Up the Application if Needed

If the sender was previously blocked, but ACK processing frees buffer space or reopens the window, generate a write event.

2. HandlePayload

This function is the core receive-side data path in Part 2. Your implementation should handle the following:

(a) Validate the Incoming Sequence Range

Before merging payload into the receive buffer, verify that the segment lies within the acceptable receive window. In particular, reject the payload that is completely old or beyond the current receive window. Such data should not be merged into the buffer.

(b) Insert Payload into the Receive Buffer

Insert the payload into the receive buffer using the incoming sequence number.

(c) Advance rcv_nxt Only for Contiguous Data

After insertion, update rcv_nxt only up to the first byte that is missing. This is one of the most important receiver-side properties of TCP: the receiver must acknowledge only the next in-order byte expected.

(d) Update rcv_wnd

After receive buffer state changes, update the currently available receive window. This value will later be advertised back to the sender through outgoing ACK packets.

(f) Notify the Application When New Readable Data Appears

If the newly received data makes additional contiguous bytes available to the application, register a read event so the socket becomes readable.

(g) Return Whether Immediate ACK is Needed

If the payload was merged cleanly and rcv_nxt advanced normally, a regular aggregated ACK may be enough. If there is a loss or an out-of-order arrival, the function should return a value indicating that an immediate ACK is preferable.

You can find detailed instructions for implementing these logics in the skeleton code!

- tcp_out.c

1. MakeTCPPacket

This function constructs outgoing TCP packets for existing flows. For Part 2, you should understand that this function must now support data transmission.

Your logic should:

- Attach payload after the TCP header
- Update retransmission timer state for retransmittable packets
- Advertise the current receive window
- Update sender-side sequence state consistently

Remember:

- payload advances `snd_nxt` by its length
- SYN consumes one sequence number
- FIN consumes one sequence number

2. *ProcessSendBuffer*

This is the key send-side data function. Your implementation should:

(a) Identify the Next Unsnt Byte

Use `snd_nxt` and `sndbuf->head_seq` to compute the correct offset into the send buffer.

(b) Compute Buffered Data Length

Determine how many unsent bytes remain from that point onward.

(c) Compute the Current Sending Limit

Calculate the amount of data that may be sent right now using both:

- flow control (`peer_wnd`)
- congestion control (`cwnd`)

Again, you can find more detailed instructions in the code!

- **timer.c**

1. *ProcessRetransmissionTimeout*

This function handles what the sender should do when an RTO occurs. Your implementation should handle the following:

(a) Increase the Retransmission Timeout Value

When a timeout occurs, the sender should increase its RTO to prevent aggressive retransmissions. In ESTABLISHED and later states, use the RTT estimator together with exponential backoff. This is important because repeated timeouts usually indicate ongoing loss or congestion, so the sender should become more conservative.

(b) Update `snd_nxt` for retransmission

9. How to Test My Code?

Testing your code is similar to what you did in part 1. The only difference is that you have to transfer a file between the server and the client. You can create any file and test with your implementation. We give you a simple Python script to generate a text file. You can use it or generate your own file.

After implementing the eTCP stack for part 2, go to the **etcp/src** directory and run **make**. Then move to the **example/part2** directory and run **make** again. This will build the ETCP_server and ETCP_client applications for part 2.

[1] eTCP Client - Linux Server

Assume you are in the **example/part2** directory of machine 1. To run the eTCP client, first open **ETCP.conf** and modify the *ip* field to match your IP address (the designated IP address for the machine you are trying to run the eTCP application on).

Next, open the **run_ETCP_client.sh** script.

For `argv[1]`, enter the IP address of the other machine running the Linux server and the name of the file you want to receive, followed by `'/'`.

For `argv[2]`, enter your designated port number, which is the port number of the remote server you want to connect to.

For `argv[3]`, enter the number of concurrent flows to create. For example, if you enter 5, the client will establish 5 connections and immediately close them. The number of concurrent flows must not exceed 5.

After completing these steps, open a terminal and execute:

```
$/run_ETCP_client.sh
```

Make sure that the server (`linux_server`) is already running on the other machine before starting the eTCP client. To run a Linux server on the other machine, go to **linux_app/part2**. Then, compile the code. Also, make sure that you have the requested file from `eTCP_client` in the directory.

```
$ gcc linux_server.c -o linux_server
```

Find **run_linux_server.sh** and set the application's arguments. Then run it on the terminal.

```
$/run_linux_server.sh
```

After the file transfer is complete, you can find the received file in the directory where you ran the eTCP client. Use Linux tools such as `diff` to verify that the file was transmitted correctly. The argument after the `-o` option in ***run_ETCP_client.sh*** specifies the name of the output file.

[2] eTCP Server - Linux Client

Open ***run_ETCP_server.sh*** and set `argv[1]` to the port number that the eTCP server will listen on. In **example/part2**, there is an additional directory named ***www_home***. By default, the eTCP server looks for requested files in ***www_home*** and serves files from that directory. Therefore, place the file you want the Linux client to request inside ***www_home***. After starting the eTCP server on machine 1, move to machine 2 and go to **linux_app/part2**. Compile `linux_client.c`, then open ***run_linux_client.sh***.

Set the arguments as follows:

- `argv[1]`: the IP address of the remote eTCP server
- `argv[2]`: the port number of the eTCP server
- `argv[3]`: the number of concurrent client flows to create
- `argv[4]`: the name of the file to request from the eTCP server

Make sure that the requested file is located in the ***www_home*** directory on the machine running eTCP server.

[3] eTCP - eTCP Testing for Debugging

For additional debugging in Part 2, you may run eTCP on both of your designated machines and test file transfer directly between two eTCP endpoints. To support this, separate eTCP-to-eTCP debugging ports have been assigned and listed in the spreadsheet. You may use those ports for testing. This setup is not the main required test configuration, but it is useful when you want to debug your own implementation more directly without involving the Linux TCP stack.

To test in this mode, run the eTCP server on one machine and the eTCP client on the other machine.

First, on both machines, open ***ETCP.conf*** and make sure that the `ip` field matches the designated IP address of that machine. Then set the port number in the run scripts to the eTCP-to-eTCP debugging port assigned in the spreadsheet.

On the machine running eTCP server, place the file to be served in the ***www_home*** directory so that the client can request it. Then start the server. On the other machine, open ***run_ETCP_client.sh*** and set the server IP address, port number, number of concurrent flows, and requested file name as usual. After that, run the client.

[4] Simulate Harsh Conditions

To simulate harsh conditions in Part 2, you can modify `drop_ack_rate` and `drop_data_rate` in the **ETCP.conf** file.

- `drop_data_rate`
Randomly drops outgoing data packets with the given probability. This is useful for testing retransmission and reliable data delivery during file transfer.
- `drop_ack_rate`
Randomly drops outgoing pure ACK packets with the given probability. This is useful for testing whether data transfer still completes correctly when acknowledgments are lost.

Choose these rates from 0 to 10, and check your implementation survives under harsh network conditions.

10. Grading Criteria

Your code should be built with the given Makefile. If not, we cannot give you a point!

- Integrity of the Submission file - 5pts
 - Submission file format, all required files
- Content of readme.pdf - 5pts
- Correct connection management implementation(part 1) - 40pts
 - Active Open - 10pts
 - Active Close - 10pts
 - Passive Open - 10pts
 - Passive Close - 10pts
- Reliable Data Transfer and Flow Control(part 2) - 50pts
 - 1KB text test - 5pts
 - 1MB text test - 10pts
 - 10MB text test - 15pts
 - 100MB text test - 20pts

11. Collaboration Policy:

- You can discuss with other students, but you should write the code on your own – you should NOT look at someone else’s code nor show/give your code to anyone else.
- Please stay away from any kind of plagiarism. If you are not sure about anything, ask TAs or the instructor.

12. Submission

Submit `tcp_in.c`, `tcp_out.c`, `timer.c`, and `readme.pdf` in a gzipped tar file format. Do NOT submit any other files. Write (1) your detailed implementation strategy, (2) how you tested your programs, (3) any bugs in the current code, (4) all collaborators in `readme.pdf`. The gzipped tar file should be “`{YourID_without_dash}_{YourNameInEnglish}_assign3`”. If your student ID is 2024-12345 and your name is “Haechan Kim”, then it should be

202412345_HaechanKim_assign3. Note that you should use your student ID without the dash (-).

Assume you have the files in the current directory:

```
$ mkdir 202412345_HaechanKim_assign3
$ cp tcp_in.c tcp_out.c timer.c readme.pdf \ 202412345_HaechanKim_assign3
$ tar zcf 202412345_HaechanKim_assign3.tar.gz \ 202412345_HaechanKim_assign3
```

**** Congestion Control**

- How to test Congestion Control

Your goal here is to verify that your sender application correctly updates its congestion-control state in response to network feedback.

In particular, you should check the following two situations:

- three duplicate ACKs
- retransmission timeout (RTO)

These are the two main loss signals that your sender should handle according to the congestion-control rules described in the lecture slides (**please refer to slide 49 of 8.TCP&CC lecture note**).

1. What to observe

Your congestion-control implementation should update at least the following variables correctly:

- cwnd
- ssthresh

You should confirm that these variables change at the correct time under the following situations:

Case A: Three Duplicate ACKs

When packet loss is inferred from three duplicate ACKs, the sender should:

- retransmit the missing data immediately,
- reduce ssthresh,
- adjust cwnd for fast retransmit / fast recovery,
- leave fast recovery when a new ACK arrives that advances the acknowledgment number.

Case B: Retransmission Timeout

When loss is inferred from a timeout, the sender should react more conservatively than in the duplicate-ACK case. In particular, the sender should:

- back off the retransmission timeout,
- reduce ssthresh,
- reset cwnd to a small value,
- and retransmit beginning from the first unacknowledged byte.

2. Test environment

Testing your implementation is the same as what you did in part 2.

A typical setup is:

- Machine 1: eTCP server under test (the sender)
- Machine 2: peer application (Linux Client that receives data)

Your congestion-control logic is implemented at the sender, so the most important machine to observe is the one running the eTCP sender.

3. Test hooks provided in the configuration file

For controlled congestion-control testing, the project provides the following fault-injection options in the configuration file:

- `drop_data_nth`: intentionally drops the n th outgoing data packet. This is useful for creating a deterministic loss event, for example, to trigger three duplicate ACKs in a controlled way.
- `drop_data_rate`: randomly drops outgoing data packets according to the configured rate. This is not deterministic, but it is useful for stress-testing your implementation under more general loss conditions, where three duplicate ACKs and retransmission timeouts may both occur.

4. Testing three duplicate ACK behavior

To test the duplicate-ACK case, create a situation in which exactly one data segment is lost while later out-of-order segments still arrive at the receiver. This causes the receiver to repeatedly acknowledge the same missing sequence number, producing duplicate ACKs. When this happens, you should observe the following pattern in the sender trace:

- duplicate ACKs start arriving,
- the duplicate ACK counter reaches 3,
- fast retransmit is triggered,
- ssthresh is reduced,
- cwnd is adjusted for recovery,
- after a new ACK arrives, the sender exits recovery.

5. Testing retransmission timeout behavior

To test timeout behavior, create a situation in which the sender does not receive the ACK progress needed to advance `snd_una`. This should eventually trigger a retransmission timeout. When this happens, you should observe the following pattern:

- the sender stops making ACK-driven progress,
- retransmission timeout occurs,
- `ssthresh` is reduced,
- `cwnd` is reset to a small value,
- retransmission starts again from the first unacknowledged byte.

Compared with the three-duplicate-ACK case, this reaction should be more conservative.

6. Logging congestion control trace

To debug and validate your congestion-control implementation, you are encouraged to add sender-side trace logs at important state-transition points in your code. At a minimum, you should log events around the following situations:

- when `cwnd` grows during slow start
- when `cwnd` grows during congestion avoidance
- when a duplicate ACK is detected
- when fast retransmit is triggered
- when additional duplicate ACKs are processed during fast recovery
- when the sender exits recovery
- when a retransmission timeout occurs

A useful trace line should include enough information to explain how the sender state changed. For example, your log may include:

- timestamp
- congestion-control event name
- ACK sequence number
- `snd_una`
- `snd_nxt`
- `cwnd`
- `ssthresh`

The exact format is up to you. You may use the logging functions implemented in `etcp/src/cc_trace.c`. Please briefly explain your congestion-control trace format in your README.

- Bonus policy for Congestion Control

Successfully implementing congestion control in eTCP gives an extra 30% credit on top of your Part 1 and Part 2 scores. For example, if your Part 1 + Part 2 score is 100 and your congestion control implementation is correct, your total score becomes 130. Congestion-control grading will focus on whether your sender shows the correct Reno-style behavior in the following cases:

- slow start (20%)
- congestion avoidance (20%)
- three duplicate ACKs (30%)
- retransmission timeout (30%)